



CLAUDE LEARNING SKOOL

How to Use Less Tokens

Cut Claude costs by 50–90% without losing quality

TOKENS & PRICING

📄 TOKEN BASICS

1 token ≈ 4 chars or ¾ word

1K tokens ≈ 750 words ≈ 1.5 pages

Input System + messages + tools + files

Output 5× the price of input

Output is the expensive side. Control its length aggressively.

💰 PRICE / 1M TOKENS

Opus 4.6 \$5 in / \$25 out

Sonnet 4.6 \$3 in / \$15 out

Haiku 4.5 \$1 in / \$5 out

Cached read **-90%** (0.1× input)

Batch API **-50%** in & out

★ THE SAVINGS STACK

1. Right-size the model **-60%**
2. Cache stable content **-90%**
3. Tighten prompts **-30%**
4. Cap outputs **-40%**
5. Batch offline jobs **-50%**

Stack them: savings multiply, not add. 3–4 combined = 80–95% cut.

THE 5 TACTICS THAT SAVE TOKENS

◆ 1. PICK THE SMALLEST MODEL THAT WORKS

USE HAIKU FOR

Classification & tagging

Extraction from structured text

Short rewrites, routing, batch jobs

RESERVE OPUS FOR

Architecture & complex code

Long-horizon agentic tasks

Hard reasoning, research synthesis

Rule: Start on Sonnet. Downgrade to Haiku for repetitive work. Upgrade to Opus only when evals prove you need it.

⚡ 2. PROMPT CACHING

Mark stable content with `cache_control`

Cache system prompts, long docs, tool schemas

5-min TTL: write 1.25×, read 0.1×

1-hour TTL: write 2×, read 0.1×

Breakeven: 2+ hits on same block

✂️ 3. SHORTEN YOUR PROMPTS

Cut pleasantries — no "please", "thanks"

One example beats ten — minimal few-shot

Reference, don't dump content

State each rule once — no overlap

Test: Cut your prompt 30% and re-run evals. Quality usually holds.

⏪ 4. CONTROL OUTPUT LENGTH

`max_tokens` — hard cap on response

`stop_sequences` — stop at marker

Ask concise: "Answer in <50 words"

Specify format: "JSON only, no prose"

Prefill with `{` to skip preamble

⏮️ 5. TRIM YOUR CONTEXT

Extract the relevant function, not whole file

Strip imports, licenses, boilerplate

Summarize old conversation turns

RAG instead of dumping the full corpus

Drop unused tools — schemas cost tokens

PRODUCT & SCALE TIPS



CLAUDE CODE HYGIENE

- `/compact` Compress long sessions
- `/clear` Reset between tasks
- `/model` Drop to Haiku for grunt work
- `@file` Attach specific files only
- `CLAUDE.md` Keep tight — loads every session



BATCH API & THINKING

Batch: up to 100K reqs, 50% off, <24h
Perfect for evals, pipelines, overnight jobs

Thinking tokens = output (expensive)
Opus/Sonnet 4.6: use `effort` (adaptive)
Off for classification & simple rewrites



MEASURE BEFORE YOU OPTIMIZE

Log `usage` on every response

Fields: `input_tokens` , `output_tokens` , `cache_read` , `cache_creation`

Token Counting API previews cost
Console Workbench shows live counts
80/20: one bad prompt drives most cost

DON'T DO THIS



COMMON TOKEN WASTERS

IN PROMPTS

- BAD** Pasting whole files "just in case"
- BAD** Re-sending docs every turn vs caching
- BAD** Using Opus for tagging/classification
- BAD** "Explain in detail" when you need the answer

IN CONVERSATIONS

- BAD** 50-turn chats without `/compact`
- BAD** Asking Claude to "repeat the full file"
- BAD** Thinking enabled on a one-line task
- BAD** Forgetting `max_tokens`



GOLDEN RULES

- Measure first** — log usage on every call
- Cache what's stable**
- Match model to task**
- Control the output** (5× input price)
- Keep context clean**

DEEP DIVE: CLAUDE CODE IN THE TERMINAL

/ TOKEN-SAVING SLASH COMMANDS

/compact	Summarizes conversation, frees context
/clear	Full reset; use when switching task
/cost	Show session token + \$ spend
/model haiku	Switch to cheapest model mid-session
/init	Build a tight CLAUDE.md
/config	Toggle auto-compact, verbose

Workflow: one task per session. /cost at end, /clear before next.

>_ CLI FLAGS THAT SAVE TOKENS

```
# Cheap one-shot, no history
claude -p "refactor x.py" \
  --model haiku # Scope model per task
claude --model sonnet # Live token usage
claude -p "..." --verbose
```

-p (print) mode skips the interactive loop: no history carryover, no wasted tokens.

≡ SLIM YOUR CLAUDE.MD

- Loads **every turn** — every word costs
- Cut generic coding rules Claude already knows
- Keep project-specific paths & "never do X"
- Subfolder CLAUDE.md loads only when entered
- Use @import for files loaded on demand
- Target:** under 50 lines. Longer? Split or prune.

DEEP DIVE: API EFFICIENCY

⚡ PROMPT CACHING — FULL EXAMPLE

PYTHON

```
import anthropic
client = anthropic.Anthropic()
resp = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=500,
    system=[{"type": "text", "text": "long docs", "cache_control": {"type": "ephemeral"}}],
    messages=[{"role": "user", "content": "q"}]
)
print(resp.usage)
```

WHAT TO CACHE

- System prompt** — role, rules, format
- Long docs** — books, codebases, policies
- Tool schemas** — stable tool definitions
- Few-shot examples**

RESPONSE USAGE FIELDS

cache_creation_input_tokens
cache_read_input_tokens — 90% off
input_tokens — not cached
output_tokens — response

Minimum cached block: 1,024 tokens (Sonnet/Opus), 2,048 (Haiku). Below that, no discount.

→ PREFILL TRICKS

```
# Force JSON output
messages=[{"role": "user", "content": "q"}, {"role": "assistant", "content": "{"}]
# Force bullet list
{"role": "assistant", "content": "- "}
```

Saves 50–200 output tokens per call by killing "Sure, here you go..."

⌂ STREAMING & STOPS

- Streaming same cost, but you can abort early
- Client-side abort when enough received
- stop_sequences=["\n\n"] — blank line
- stop_sequences=["</output>"] — at tag
- Inspect stop_reason : end_turn vs max_tokens

⚙️ TOOL USE & AGENTS

- Minimize tool count — schemas cost tokens
- Tight descriptions, no examples unless needed
- Paginate — no 10K rows in tool_result
- Summarize results before feeding back
- Cache tool schemas when stable